

POX controller and Layer 2 switching

Objective: To learn how to run the POX controller and analyse the basic switch working. To see how few ports can be blocked.

- Install and run POX + Mininet.
- Run a simple learning switch controller (POX) against a one-switch two-host topology.
- Inspect flows, capture packets, and debug.

In newer versions of Mininet, POX is installed by default. Its folder is inside the Mininet folder. Check the available files in the pox folder and explore them.

You can also download the POX controller inside the Mininet folder by using the following command.

```
git clone https://github.com/noxrepo/pox
```

Check the folder of pox inside the mininet folder.

```
cd pox
```

Simple topology (learning switch)

Topology: 1 switch (s1), 3 hosts (h1, h2,h3).

Controller: POX learning switch.

Running POX Controller

Open a terminal in Ubuntu and type the following command.

(Locate your pox.py and provide the correct paths)

```
sudo ~/mininet/pox/pox.py forwarding.l2_pairs
```

or

```
./pox.py log.level --DEBUG forwarding.l2_learning
```

```
saim@saimvm:~$ sudo ~/mininet/pox/pox.py forwarding.l2_pairs
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
INFO:forwarding.l2_pairs:Pair-Learning switch running.
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7
.8 3.9
WARNING:version:You're running Python 3.10.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-01 3] connected
```

****Go to the pox folder directly if the above relative paths are not working. We can run pox.py by typing ./pox.py FILENAME**

Running a topology in Mininet

Open another terminal and type the following command to load the topology. The following topology will load. We want to see how the switch will learn and make pairs in this lab.

```
root@saimvm:/home/saim/mininet# sudo mn --topo single,3 --mac --switch ovsk --c
ontroller remote
*** Creating network
*** Adding controller
Unable to contact the remote controller at 127.0.0.1:6653
Connecting to remote controller at 127.0.0.1:6633
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
```

** Port 6633 is the default port for the POX controller

Now in this terminal, type.

Xterm h1 h2 h3

The terminal windows of h1, h2, and h3 will open. In the terminals of h2 and h3, type the following command and leave them open.

Tcpdump

Now, in the terminal of h1, type the following command and see the output in the terminal window of h2 and h3.

Ping 10.0.0.2

Ping 10.0.0.3

[illegible]

To check the flows, type the following command.

dpctl dump-flows

```
mininet> dpctl dump-flows
*** s1 -----
  cookie=0x0, duration=122.154s, table=0, n_packets=40, n_bytes=3696, dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:"s1-eth2"
  cookie=0x0, duration=122.154s, table=0, n_packets=41, n_bytes=3738, dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
  cookie=0x0, duration=54.435s, table=0, n_packets=56, n_bytes=5320, dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03 actions=output:"s1-eth3"
  cookie=0x0, duration=54.435s, table=0, n_packets=57, n_bytes=5362, dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
mininet>
```

Check the following command as well from the Mininet or host pc shell.

```
sudo ovs-ofctl dump-flows s1
```

The above two commands will show you the flow entries.

Type the following command to check ARP.

h1 arp

```
mininet> h1 arp
Address HWtype HWaddress Flags Mask Iface
10.0.0.3 ether 00:00:00:00:00:03 C h1-e
10.0.0.2 ether 00:00:00:00:00:02 C h1-e
mininet>
```

To clear all flow tables on all switches, enter the Mininet command:

```
mininet> dpctl del-flows
```

Now, check again the data flows using the previous command.

Firewall program for POX controller:

Read the following code and try to understand it. A simple example for blocking arbitrary TCP ports:

```
"""
Block TCP ports
Save as ext/blocker.py and run along with L2_Learning.
You can specify ports to block on the commandline:
./pox.py forwarding.L2_Learning blocker --ports=80,8888,8000
Alternatively, if you run with the "py" component, you can use the CLI:
./pox.py forwarding.L2_Learning blocker py
...
POX> block(80, 8888, 8000)
"""

from pox.core import core
# A set of ports to block
block_ports = set()
```

```
def block_handler (event):  
    # Handles packet events and kills the ones with a blocked port number  
  
    tcp = event.parsed.find('tcp')  
  
    if not tcp: return # Not TCP  
  
    if tcp.srcport in block_ports or tcp.dstport in block_ports:  
        # Halt the event, stopping L2_Learning from seeing it  
        # (and installing a table entry for it)  
  
        core.getLogger("blocker").debug("Blocked TCP %s <-> %s",  
                                         tcp.srcport, tcp.dstport)  
  
        event.halt = True  
  
def unblock (*ports):  
    block_ports.difference_update(ports)  
  
def block (*ports):  
    block_ports.update(ports)  
  
def launch (ports = ''):  
    # Add ports from commandline to list of ports to block  
  
    block_ports.update(int(x) for x in ports.replace(", ", " ").split())  
  
    # Add functions to Interactive so when you run POX with py, you  
    # can easily add/remove ports to block.  
  
    core.Interactive.variables['block'] = block  
    core.Interactive.variables['unblock'] = unblock  
  
    # Listen to packet events  
  
    core.openflow.addListenerByName("PacketIn", block_handler)
```

Save this code in a separate file as 'blocker.py' and save it under /pox/ext/ folder.

Now, use the following command to run the PoX controller and add the ports to block with the command as follows.

```
root@saim-virtual-machine:/home/saim/mininet/pox# ./pox.py forwarding.l2_learning blocker --  
ports=80,8888,8000,22,7
```

Now, the controller will run with this blocker file and will try to block the ports.

In a separate window, the same as the previous load mininet topology. Now, try to use SSH from host 1 to host 2 or host 3 and observe the output.

Exercise:

Create a topology as two switches and 2 hosts per switch. Now run the topology using the similar command in mininet. Let the pox controller run in a separate window. It must show that all the switches are connected. Now open two host terminals and run tcpdump on one host terminal, and ping it from the terminal of the first host. See the flow table and analyse the tcpdump traffic.

Reading:

This is a good source to expand further your knowledge and understanding of the POX controller.

<https://noxrepo.github.io/pox-doc/html/#id4>